

Report on Stream Learning Experiments using River and CapyMOA

Osiris Yetna - Matthieu Kaeppelin

25/10/2025

1 Introduction

This experiment aims to compare the performance of several online learning algorithms implemented in two major stream learning frameworks: **River** (Python) and **CapyMOA** (Java-based, inspired by MOA). The comparison covers multiple learning paradigms, including classification, regression, anomaly detection, and drift detection, across different datasets. The goal is to evaluate the behavior, performance, and adaptability of these algorithms in streaming data scenarios, considering accuracy, computational efficiency, and adaptability to concept drift.

2 Methodology

The following steps were performed:

- **Setup:** Both River and CapyMOA libraries were downloaded from their official websites.
- **Selected tasks and datasets:**
 - Regression: **Phishing**
 - Anomaly Detection: **CovType**
 - Classification / Concept Drift: **ElectricityTiny**
- **Selected models:**
 - Regression - River : **Logistic Regression**,
 - Anomaly Detection (River VS CapyMOA) : **Half-Space Tree**
 - Classification distance based (River VS CapyMOA): **KNN**
 - Classification - Tree (River VS CapyMOA) : **Hoeffding Adaptive Tree**
 - Boosting methods - CapyMOA : **OzaBoost**
 - Boosting methods and Drift - River : **ADWIN Boosting**
- **Evaluation metrics:** Accuracy, ROC AUC, PR-AUC, Recall, F1-score.

All experiments were conducted using progressive validation (predict-then-learn) to simulate real-time data streams.

3 Results and Observations

3.1 Logistic Regression (River)

Dataset: Phishing

Final Accuracy: 0.8896

River’s Logistic Regression achieved nearly 89% accuracy using stochastic gradient descent (SGD). It demonstrates good online performance for binary classification tasks with incremental learning.

3.2 Anomaly Detection (Half-Space Tree)

Table 1: Half-Space Tree results on the CovType dataset

Model	ROC AUC	PR-AUC	Recall	F1-Score
CapyMOA	0.7321	0.1681	0.5737	0.2902
River	0.8133	0.4954	0.9955	0.3636

The River Half-Space Tree clearly outperforms CapyMOA’s implementation, achieving higher ROC AUC and PR-AUC scores. Its nearly perfect recall (0.99) indicates that River’s model identifies almost all anomalies, although its moderate F1-score reveals some false positives.

3.3 K-Nearest Neighbors (KNN)

Part 1 — CapyMOA KNN

Experiments were conducted on the `ElectricityTiny` dataset with different window sizes and K values. The goal was to analyze how accuracy changes with hyperparameters and to evaluate the model’s variance and stability.

Table 2: CapyMOA KNN accuracy depending on K and window size

Window Size	10	25	50	100
$K = 5$	80.10	86.15	89.80	87.50
$K = 10$	69.00	79.35	87.25	86.55
$K = 15$	69.00	75.25	83.35	85.70
$K = 20$	69.00	70.10	82.15	84.80

Best configuration: $K = 5$, Window Size = 50, Accuracy = 89.8%.

Analysis. CapyMOA’s KNN does not use *weighted voting*: each neighbor contributes equally to the prediction. This lack of weighting causes higher variance in predictions and a strong sensitivity to the choice of K and window size. For instance, performance drops sharply as K increases, highlighting instability when hyperparameters are not tuned precisely.

Part 2 — River KNN

River’s `KNNClassifier` was evaluated with the same dataset and different K values. The implementation supports distance-weighted voting, reducing variance and improving stability across hyperparameters.

Table 3: River KNN accuracy for different K values

K	Accuracy
5	0.8409
10	0.8379
15	0.8424
20	0.8329

Analysis. The results are more stable compared to CapyMOA’s KNN. Accuracy varies only slightly with K , showing that distance weighting makes River’s KNN more robust to parameter changes and data noise. This leads to smoother learning curves and reduced variance in performance.

Part 3 — Comparative Conclusion

- **Best CapyMOA configuration:** $K = 5$, Window=50, Accuracy = 89.8%.
- **Best River configuration:** $K = 15$, Accuracy = 84.2%.

While CapyMOA reaches slightly higher peak accuracy, its performance is much more sensitive to hyperparameter tuning. River’s KNN maintains consistent accuracy even with varying K , demonstrating better stability and generalization properties. Thus, in practice, River’s KNN is preferable for evolving data streams where hyperparameters cannot be fine-tuned dynamically.

3.4 Hoeffding Adaptive Tree (HAT)

Table 4: Hoeffding Adaptive Tree performance

Framework	Accuracy
CapyMOA	84.10%
River	83.48%

Both frameworks achieve similar performance, demonstrating that River’s implementation matches the reliability of CapyMOA’s reference model for adaptive decision trees.

3.5 Boosting Methods (OzaBoost vs ADWIN Boosting)

Table 5: Boosting method comparison

Model	Accuracy
CapyMOA OzaBoost	93.3%
River ADWIN Boosting	84.8%

CapyMOA’s OzaBoost achieves higher accuracy through gradient boosting with Hoeffding Trees and adaptive weighting. River’s ADWIN Boosting uses online drift detection to adjust learner weights dynamically. Although less accurate here, it demonstrates robustness and adaptability in changing environments.

4 Global Discussion

CapyMOA occasionally achieves higher accuracy thanks to its optimized Java implementations, while River offers greater flexibility and modularity through its Python-based design. Both frameworks efficiently adapt to data streams and concept drift. For imbalanced datasets, metrics such as ROC-AUC and F1-score prove more informative than simple accuracy. Additionally, River’s intuitive syntax and architecture make it more accessible for research and experimentation.

5 Conclusions

Overall, both River and CapyMOA are powerful stream learning frameworks. River excels in anomaly detection, model flexibility, and stability, while CapyMOA’s ensemble methods (like OzaBoost) can achieve high accuracy in certain datasets, though they do not explicitly detect or adapt to sudden concept drift as River’s ADWIN Boosting does. For research and prototyping, River’s simplicity and Python-based ecosystem make it particularly appealing.